Maximum Sum of 3 Non-Overlapping Subarrays

Problem

Given an integer array `nums` and an integer `k`, find three non-overlapping contiguous subarrays, each of length `k`, such that their total sum is maximum.

Example

```
nums = [1, 2, 1, 2, 6, 7, 5, 1]
k = 2
```

One possible selection is:

```
[1, 2] | [6, 7] | [5, 1]
```

Their sums are:

```
3 + 13 + 6 = 22
```

The starting indices are:

```
[0, 4, 6]
```

---

1. Logic Building

We need to choose exactly three non-overlapping subarrays:

```
LEFT | MIDDLE | RIGHT
```

Each subarray has exactly length `k`.

Therefore:

```
best left window
+
current middle window
```

```
+
best right window
```

The main question is:

> For every possible middle window, what is the best valid window before it and after it?

---

2. Step 1: Create All Fixed-Length Window Sums

Given:

```
nums = [1, 2, 1, 2, 6, 7, 5, 1]
k = 2
```

All windows of length 2 are:

```
Window 0 → nums[0:2] → [1, 2] → 3
Window 1 → nums[1:3] → [2, 1] → 3
Window 2 → nums[2:4] → [1, 2] → 3
Window 3 → nums[3:5] → [2, 6] → 8
Window 4 → nums[4:6] → [6, 7] → 13
Window 5 → nums[5:7] → [7, 5] → 12
Window 6 → nums[6:8] → [5, 1] → 6
```

Therefore:

```
window_sum = [3, 3, 3, 8, 13, 12, 6]
```

Number of windows:

```
n - k + 1
```

For:

```
n = 8
k = 2
```

we get:

```
8 - 2 + 1 = 7
```

windows.

---

3. Window Index vs Normal Array Index

This is the most important concept.

Original array:

```
Index:  0  1  2  3  4  5  6  7
Value:  1  2  1  2  6  7  5  1
```

Because:

```
k = 2
```

we have:

```
window[i] = nums[i : i + k]
```

Therefore:

```
Window 0 → nums[0:2] → indices 0,1 → sum 3
Window 1 → nums[1:3] → indices 1,2 → sum 3
Window 2 → nums[2:4] → indices 2,3 → sum 3
Window 3 → nums[3:5] → indices 3,4 → sum 8
Window 4 → nums[4:6] → indices 4,5 → sum 13
Window 5 → nums[5:7] → indices 5,6 → sum 12
Window 6 → nums[6:8] → indices 6,7 → sum 6
```

Mapping:

```
Window Index:  0      1      2      3      4      5      6
Window Sum:    3      3      3      8     13     12      6

Array Indices: 0,1    1,2    2,3    3,4    4,5    5,6    6,7
```

The important formula is:

```
window[i] = nums[i : i + k]
```

For example:

```
window[3] = nums[3:5]
```

This means:

```
nums[3] and nums[4]
```

Therefore:

```
window index 3
```

represents the original array subarray:

```
[2, 6]
```

---

4. Choosing a Middle Window

Suppose:

```
middle = 3
```

Then:

```
window[3] = nums[3:5] = [2, 6]
```

It occupies normal array indices:

```
3, 4
```

We need:

```
LEFT | MIDDLE | RIGHT
```

Left Side

The left window must not overlap with array indices 3 or 4.

Check:

```
Window 2 → indices 2,3 → overlaps ❌
Window 1 → indices 1,2 → valid ✅
Window 0 → indices 0,1 → valid ✅
```

Therefore:

```
valid left windows = [0, 1]
```

Right Side

The right window must not overlap with array indices 3 or 4.

Check:

```
Window 4 → indices 4,5 → overlaps ❌
Window 5 → indices 5,6 → valid ✅
Window 6 → indices 6,7 → valid ✅
```

Therefore:

```
valid right windows = [5, 6]
```

So:

```
LEFT       MIDDLE       RIGHT
[0,1]       [3]         [5,6]
```

---

5. Deriving the Boundary Formula

Suppose:

```
middle = i
k = window length
```

The middle window is:

```
nums[i : i + k]
```

It occupies:

```
i, i + 1, ..., i + k - 1
```

Left Boundary

The latest possible left window starts at:

```
i - k
```

Therefore:

```
left window index <= i - k
```

Right Boundary

The earliest possible right window starts at:

```
i + k
```

Therefore:

```
right window index >= i + k
```

So the formula is:

```
left  <= middle - k
```

```
right >= middle + k
```

For:

```
middle = 3
k = 2
```

we get:

```
left <= 3 - 2 = 1
right >= 3 + 2 = 5
```

Therefore:

```
left windows  = [0, 1]
middle        = [3]
right windows = [5, 6]
```

---

6. What Do We Precompute?

For every window, we want to quickly know:

```
What is the best window from the beginning up to this position?
```

and:

```
What is the best window from this position until the end?
```

So we create:

```
left_best[i]
right_best[i]
```

These arrays store the INDEX of the best window.

They do not store the actual sum.

---

7. Building `left_best`

Definition:

```
left_best[i]
```

stores the index of the maximum-sum window among:

```
window[0 ... i]
```

Given:

```
window_sum = [3, 3, 3, 8, 13, 12, 6]
```

Scan from left to right.

```
i = 0
range = [3]
best index = 0
```

```
i = 1
range = [3, 3]
best index = 0
```

When sums are equal, we keep the earlier index.

```
i = 2
range = [3, 3, 3]
best index = 0
```

```
i = 3
range = [3, 3, 3, 8]
best index = 3
```

```
i = 4
range = [3, 3, 3, 8, 13]
```

```
best index = 4
```

```
i = 5
range = [3, 3, 3, 8, 13, 12]
best index = 4
```

```
i = 6
range = [3, 3, 3, 8, 13, 12, 6]
best index = 4
```

Therefore:

```
left_best = [0, 0, 0, 3, 4, 4, 4]
```

For example:

```
left_best[2] = 0
```

means:

> Among windows `0, 1, 2`, the best window is window `0`.

And:

```
left_best[5] = 4
```

means:

> Among windows `0, 1, 2, 3, 4, 5`, the best window is window `4`.

The code:

```
left_best = [0] * m

best_index = 0

for i in range(m):

    if window_sum[i] > window_sum[best_index]:
        best_index = i
```

```
    left_best[i] = best_index
```

---

8. Building `right_best`

Definition:

```
right_best[i]
```

stores the index of the maximum-sum window among:

```
window[i ... end]
```

We scan from right to left.

Given:

```
window_sum = [3, 3, 3, 8, 13, 12, 6]
```

Start from the right:

```
i = 6
range = [6]
best index = 6
```

```
i = 5
range = [12, 6]
best index = 5
```

```
i = 4
range = [13, 12, 6]
best index = 4
```

```
i = 3
range = [8, 13, 12, 6]
```

```
best index = 4
```

```
i = 2
best index = 4
```

```
i = 1
best index = 4
```

```
i = 0
best index = 4
```

Therefore:

```
right_best = [4, 4, 4, 4, 4, 5, 6]
```

The code:

```
right_best = [0] * m

best_index = m - 1

for i in range(m - 1, -1, -1):

    if window_sum[i] >= window_sum[best_index]:
        best_index = i

    right_best[i] = best_index
```

---

9. Try Every Possible Middle Window

For every possible middle window:

```
middle = i
```

we choose:

```
left = left_best[i - k]
```

and:

```
right = right_best[i + k]
```

Then:

```
total =
window_sum[left]
+
window_sum[middle]
+
window_sum[right]
```

Why?

Because:

```
left <= i - k
```

and:

```
right >= i + k
```

guarantee that the three windows are non-overlapping.

---

10. Complete Dry Run

We have:

```
window_sum = [3, 3, 3, 8, 13, 12, 6]
k = 2
```

And:

```
left_best  = [0, 0, 0, 3, 4, 4, 4]
right_best = [4, 4, 4, 4, 4, 5, 6]
```

Middle = 2

Middle:

```
window[2] = 3
```

Left:

```
left_best[2 - 2]
= left_best[0]
= 0
```

Right:

```
right_best[2 + 2]
= right_best[4]
= 4
```

Total:

```
window_sum[0] + window_sum[2] + window_sum[4]
```

```
3 + 3 + 13 = 19
```

Selected windows:

```
Window 0 | Window 2 | Window 4
```

Original array:

```
[1, 2] | [1, 2] | [6, 7]
```

Middle = 3

Middle:

```
window[3] = 8
```

Left:

```
left_best[3 - 2]
= left_best[1]
= 0
```

Right:

```
right_best[3 + 2]
= right_best[5]
= 5
```

Total:

```
window_sum[0] + window_sum[3] + window_sum[5]
```

```
3 + 8 + 12 = 23
```

Selected windows:

```
Window 0 | Window 3 | Window 5
```

Original array indices:

```
[0,1] | [3,4] | [5,6]
```

Original subarrays:

```
[1, 2] | [2, 6] | [7, 5]
```

Total:

```
3 + 8 + 12 = 23
```

Middle = 4

Middle:

```
window[4] = 13
```

Left:

```
left_best[4 - 2]
= left_best[2]
= 0
```

Right:

```
right_best[4 + 2]
= right_best[6]
= 6
```

Total:

```
window_sum[0] + window_sum[4] + window_sum[6]
```

```
3 + 13 + 6 = 22
```

Selected windows:

```
Window 0 | Window 4 | Window 6
```

Original array:

```
[1, 2] | [6, 7] | [5, 1]
```

Total:

```
3 + 13 + 6 = 22
```

---

11. Why Do We Start the Middle Loop at `k`?

We need one complete window before the middle.

If:

```
middle < k
```

there is not enough space for a previous non-overlapping window.

Therefore:

```
for middle in range(k, ...):
```

For:

```
k = 2
```

the first possible middle is:

```
middle = 2
```

because:

```
Window 0 | Window 2
```

is valid.

---

12. Why Do We Stop at `m - k`?

We need one complete window after the middle.

Therefore:

```
middle + k < m
```

which means:

```
middle < m - k
```

So the loop is:

```
for middle in range(k, m - k):
```

For:

```
m = 7
k = 2
```

the middle indices are:

```
2, 3, 4
```

Exactly the valid possibilities.

---

13. Sliding Window to Build `window_sum`

Instead of calculating every window sum from scratch, use a sliding window.

For:

```
nums = [1, 2, 1, 2, 6, 7, 5, 1]
k = 2
```

First window:

```
[1, 2]
```

Sum:

```
3
```

Move one position right:

```
[1, 2] → [2, 1]
```

Remove:

```
1
```

Add:

```
1
```

New sum:

```
3
```

Move again:

```
[2, 1] → [1, 2]
```

Remove:

```
2
```

Add:

```
2
```

New sum:

```
3
```

The formula is:

```
new_sum = old_sum
          - nums[i-k]
          + nums[i]
```

Code:

```
window_sum = [0] * (n - k + 1)

current = sum(nums[:k])
window_sum[0] = current

for i in range(k, n):

    current += nums[i]
    current -= nums[i - k]

    window_sum[i - k + 1] = current
```

---

14. Complete Code

```
def max_sum_of_three_subarrays(nums, k):

    n = len(nums)

    # -----------------------------------
    # Step 1: Calculate every window sum
    # -----------------------------------

    window_sum = [0] * (n - k + 1)

    current = sum(nums[:k])
    window_sum[0] = current

    for i in range(k, n):

        current += nums[i]
        current -= nums[i - k]

        window_sum[i - k + 1] = current
```

```
        m = len(window_sum)

        # ------------------------------------
        # Step 2: Prefix best window
        # ------------------------------------

        left_best = [0] * m

        best_index = 0

        for i in range(m):

            if window_sum[i] > window_sum[best_index]:
                best_index = i

            left_best[i] = best_index

        # ------------------------------------
        # Step 3: Suffix best window
        # ------------------------------------

        right_best = [0] * m

        best_index = m - 1

        for i in range(m - 1, -1, -1):

            if window_sum[i] >= window_sum[best_index]:
                best_index = i

            right_best[i] = best_index

        # ------------------------------------
        # Step 4: Try every middle window
        # ------------------------------------

        answer = [-1, -1, -1]
        max_total = float("-inf")

        for middle in range(k, m - k):

            left = left_best[middle - k]
            right = right_best[middle + k]

            total = (
                window_sum[left]
                + window_sum[middle]
                + window_sum[right]
            )

            if total > max_total:

                max_total = total
```

```
            answer = [
                left,
                middle,
                right
            ]

    return answer
```

---

15. Code Dry Run

For:

```
nums = [1, 2, 1, 2, 6, 7, 5, 1]
k = 2
```

Step 1:

```
window_sum = [3, 3, 3, 8, 13, 12, 6]
```

Step 2:

```
left_best = [0, 0, 0, 3, 4, 4, 4]
```

Step 3:

```
right_best = [4, 4, 4, 4, 4, 5, 6]
```

Step 4:

For:

```
middle = 2
```

```
left = left_best[0] = 0
right = right_best[4] = 4
total = 3 + 3 + 13 = 19
```

For:

```
middle = 3
```

```
left = left_best[1] = 0
right = right_best[5] = 5
total = 3 + 8 + 12 = 23
```

For:

```
middle = 4
```

```
left = left_best[2] = 0
right = right_best[6] = 6
total = 3 + 13 + 6 = 22
```

Maximum:

```
23
```

Answer:

```
[0, 3, 5]
```

The selected subarrays are:

```
nums[0:2] = [1, 2] → 3
nums[3:5] = [2, 6] → 8
nums[5:7] = [7, 5] → 12
```

Total:

```
3 + 8 + 12 = 23
```

---

16. Important Correction About the Example

The commonly shown selection:

```
[1, 2] | [6, 7] | [5, 1]
```

has total:

```
3 + 13 + 6 = 22
```

But for:

```
nums = [1, 2, 1, 2, 6, 7, 5, 1]
k = 2
```

the better valid selection is:

```
[1, 2] | [2, 6] | [7, 5]
```

with indices:

```
[0, 3, 5]
```

and total:

```
3 + 8 + 12 = 23
```

These subarrays are non-overlapping:

```
[0,1] | [3,4] | [5,6]
```

Therefore, the maximum sum is:

```
23
```

17. Pattern Name

Fixed-Length Non-Overlapping Subarrays + Prefix/Suffix Best

The complete pattern is:

```
Fixed-length subarrays
        ↓
Sliding Window
        ↓
Create all window sums
        ↓
Prefix Best
        +
Suffix Best
        ↓
Choose middle window
        ↓
Combine:
leftBest + middle + rightBest
```

This is a combination of:

1. Sliding Window
2. Prefix Dynamic Programming
3. Suffix Dynamic Programming
4. Non-overlapping Interval Selection

---

18. General Template

For fixed-length subarrays:

```
window_sum[i] = sum of subarray starting at i
```

Then:

```
left_best[i] =
best window index in [0 ... i]
```

and:

```
right_best[i] =
best window index in [i ... end]
```

For three windows:

```
for middle in valid positions:

    left = left_best[middle - k]

    right = right_best[middle + k]

    answer =
        window_sum[left]
        + window_sum[middle]
        + window_sum[right]
```

General structure:

```
LEFT | MIDDLE | RIGHT
  ↓       ↓       ↓
best    fixed   best
left    middle  right
```

---

19. Quick Identification

When you see:

> Select multiple non-overlapping subarrays.

Ask:

Question 1

Are the subarrays fixed length?

Yes → Sliding Window
No → Kadane / Prefix Sum / DP may be needed

Question 2

Can the array be divided into:

LEFT | MIDDLE | RIGHT

If yes:

Precompute best answers on the left and right.

Question 3

Does the middle determine the valid boundaries?

For fixed length k:

left <= middle - k
right >= middle + k

Then:

Prefix Best + Current Middle + Suffix Best

is a strong candidate pattern.

---

20. Complexity

Let:

n = len(nums)

Time Complexity

Window sums:

O(n)

Prefix best:

O(n)

Suffix best:

O(n)

Middle loop:

O(n)

Total:

O(n)

Space Complexity

window_sum → O(n)
left_best → O(n)
right_best → O(n)

Total:

O(n)

---

21. Real-World Analogy

Suppose a company wants to select three non-overlapping time slots for three advertisements.

Each advertisement must run for exactly k minutes.

Each time slot has a profit:

slot 0 → profit 3
slot 1 → profit 3
slot 2 → profit 3
slot 3 → profit 8
slot 4 → profit 13
slot 5 → profit 12
slot 6 → profit 6

For every possible middle advertisement:

best advertisement before it
+
current advertisement
+
best advertisement after it

The same algorithm finds the maximum total profit.

---

22. Interview Quick Revision

Problem

Choose 3 non-overlapping subarrays of fixed length k with maximum total sum.

Pattern

Sliding Window
+
Prefix Best
+
Suffix Best

Steps

1. Calculate every length-k window sum.

2. Build left_best: best window index from 0 to i.
3. Build right_best: best window index from i to end.
4. Try every possible middle window.
5. Find:

   left = left_best[middle - k]
   right = right_best[middle + k]

6. Calculate:

   window_sum[left]

   - window_sum[middle]
   - window_sum[right]

7. Keep the maximum.

Most Important Formula

left = left_best[middle - k]
right = right_best[middle + k]

Why?

Because the middle window:

nums[middle : middle + k]

occupies:

middle ... middle + k - 1

Therefore:

left window must finish before middle

right window must start at or after middle + k

Final Pattern

Fixed-Length Subarrays
↓
Sliding Window
↓
Window Sums
↓
Prefix Best

+

Suffix Best

↓

Try Middle

↓

Best Left + Middle + Best Right

↓

Maximum Answer